The REINFORCE Algorithm

From Math to PyTorch Implementation

The Objective: Maximizing Expected Return

- In reinforcement learning, we use a neural network as the agent's "brain" with parameters θ.
- The output is a policy $\pi_\theta(a|s)$, which is the probability of taking action a in state s.
- Our ultimate goal is to maximize the expected return (total score).
- The objective function is defined as:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

- τ represents a full trajectory, and $R(\tau)$ is the total reward for that trajectory. We need to use gradient ascent: $\nabla_\theta J(\theta)$.

The Obstacle & The Log-Derivative Trick

- When calculating the derivative, we face an issue. The equation is no longer an expectation, so we cannot estimate it by sampling:

$$\nabla_\theta J(\theta) = \sum_\tau \nabla_\theta P(\tau|\theta) R(\tau)$$

- **The Magic Step:** We apply the Log-Derivative Trick from basic calculus:

$$\nabla x = x \nabla \log x$$

- By applying this, the formula perfectly transforms back into an expectation format:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \log P(\tau|\theta) R(\tau) \right]$$

Stripping the Environment Variables

- The probability of a full trajectory is complex. Using logarithms turns multiplication into addition:

$$\log P(\tau|\theta) = \log P(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t|s_t) + \sum_{t=0}^{T} \log P(s_{t+1}|s_t, a_t)$$

- **Crucial Insight:** The initial state $P(s_0)$ and environment transition probabilities $P(s_{t+1}|s_t, a_t)$ are independent of our network parameters θ.
- When deriving with respect to θ, these environment terms become 0 and disappear! We are only left with the policy network's output:

$$\nabla_\theta \log P(\tau|\theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

The Core REINFORCE Formula

- Plugging everything back in, we get the famous REINFORCE core formula:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau) \right]$$

- **Intuitive Understanding:**

High Return $R(\tau)$

Generates a positive gradient to **increase** the probability of those actions.

Low/Negative Return $R(\tau)$

Generates a negative gradient to **suppress** those actions.

Good behaviors are reinforced!

PyTorch Implementation: Math to Code

PyTorch optimizers minimize loss by default. To maximize our expected return, we simply add a negative sign to our mathematical objective.

A single line of code captures expectations, derivatives, and log tricks:

```
import torch

# log_probs: list of log probabilities of actions taken
# return_t: cumulative return for the trajectory

policy_loss = []
for log_prob in log_probs:
 # Core Mapping: Negative sign to convert maximization to Loss
 loss_step = -log_prob * return_t
 policy_loss.append(loss_step)

# Sum and Backpropagate
optimizer.zero_grad()
loss = torch.stack(policy_loss).sum()
loss.backward()
optimizer.step()
```